

COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE

Coal Project



Group Members:

Zain Kashif 01-135232-107
Himayat Ullah 01-135232-028
Asim Mehmood 01-135232-054

Submitted to:
Sir Adnan Jelani

Department of Computer Science
Bahria University, Islamabad

1 Abstract

This project implements a simplified Space Invaders game in 8086 assembly language for the emu8086 environment. The program uses BIOS interrupts for keyboard input, time synchronization, and text-mode rendering, while adhering to strict 8086 constraints such as limited registers, 16-bit addressing, and DOS/BIOS calling conventions. The game maintains state for player position, bullets, enemies, score, and end-game conditions using arrays in the single .COM memory segment. The main loop performs frame synchronization, input polling, entity updates, collision detection, and incremental re-drawing with an erase-previous technique to reduce flicker. The final outcome is a playable ASCII Space Invaders clone featuring movement, shooting, enemy swarm motion with boundary drops, scoring, win/lose screens, and restart/quit behavior.

2 Introduction

Problem statement: Build an interactive arcade-style game using only 8086 assembly and BIOS/DOS services.

Motivation: Demonstrate how hardware-level constraints shape program design (graphics, input, timing, memory layout).

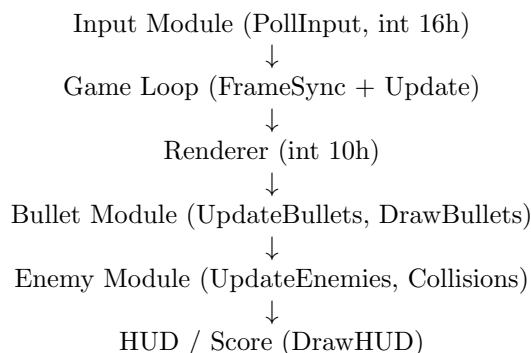
Scope: Text-mode (80x25) gameplay with player movement, multiple bullets, enemy grid, scoring, and end screens.

Objectives:

1. Apply procedures, arrays, loops, and interrupts in a non-trivial program.
2. Maintain and update game state safely.
3. Provide modular code that runs unmodified in emu8086.

3 System Design

3.1 Module / Architecture Diagram



3.2 Memory Layout (Key Data Structures)

Symbol	Type/Size	Purpose
GameState	db (1)	0=Playing, 1=Win, 2=Game Over
ExitFlag	db (1)	Exit request (ESC)
PlayerX / PlayerY	db+db (2)	Player position on 80x25 grid
LastTick	dw (2)	BIOS tick snapshot for frame sync
EnemyDir	db (1)	Swarm direction: 1 or 0FFh
EnemyMoveDelay Counter	/ dw+dw (4)	Enemy speed control
EnemyAnim	db (1)	Sprite frame toggle
Score / PrevScore	dw+dw (4)	Score and redraw cache
BulletActive/X/Y	3 arrays, 5 bytes each	Up to 5 bullets

Symbol	Type/Size	Purpose
PrevBulletActive/X/Y	3 arrays, 5 bytes each	Erase-previous cache
EnemyAlive/X/Y	3 arrays, 55 bytes each	Enemy grid state
PrevEnemyAlive/X/Y	3 arrays, 55 bytes each	Erase-previous cache

3.3 Design Decisions

1. Text-mode rendering (mode 03h) avoids graphics libraries and stays inside BIOS services.
2. Game state stored in arrays (EnemyAlive/X/Y, BulletActive/X/Y) to support multiple entities without dynamic memory.
3. Flicker control uses an erase-previous approach (Prev* arrays) before drawing the next frame.
4. Timing uses BIOS tick (int 1Ah) to keep movement stable across CPU speeds in emu8086.

4 Implementation Details

All code is contained in a single DOS `.COM` program (`org 100h`). Major procedures and their roles are listed below, with code references to `space_invaders_fixed.asm` line ranges.

Procedure	Role / Notes	Key BIOS/DOS
SetTextMode (125–129)	Set 80x25 text mode (03h).	int 10h
FrameSync (168–185)	Wait for BIOS tick change (~18.2 Hz).	int 1Ah
PollInput (190–264)	Read keys: A/D or arrows, Space, R, ESC.	int 16h
FireBullet / UpdateBullets (287–344)	Spawn bullet, move upward, deactivate at top.	–
InitEnemies (349–393)	Initialize 5x11 grid positions and alive flags.	–
UpdateEnemies (395–446)	Move swarm, drop at edges, toggle animation.	–
CheckCollisions (553–635)	Bullet vs enemy hit test, update score.	int 10h (via PutCharAt)
CheckWinLose (637–673)	Lose if enemy reaches PlayerY; win if all dead.	–
DrawHUD / DrawScoreIfChanged (678–721)	HUD strings + numeric score redraw.	int 10h
ErasePrevEntities / SyncPrevState (723–816)	Erase old positions; cache new positions.	int 10h
PutCharAt / PrintZAt / PrintNum6At (971–1048)	Low-level text output helpers.	int 10h
ShowWinScreen / ShowGameOverScreen (1055–1081)	End screens, then wait key.	int 16h + int 10h
Quit (117–120)	Return to DOS.	int 21h

4.1 Interrupts Used

1. int 10h (BIOS video): mode set (03h), cursor positioning, writing characters with attributes, teletype printing, and scroll/clear operations.
2. int 16h (BIOS keyboard): non-blocking key checks in gameplay and blocking waits on end screens.
3. int 1Ah (BIOS time): read system tick to control frame timing.
4. int 21h (DOS): program termination (AH=4Ch).

4.2 Assembly Concept Coverage

1. Procedures and stack discipline: push/pop around procedures (multiple, e.g., PollInput 190–264).
2. Arrays and indexing: BulletActive/X/Y and EnemyAlive/X/Y arrays (25–42) with DI/SI loops.
3. Nested loops: collision detection bullet loop + enemy loop (553–627).
4. Conditional branching / state machine: GameState-driven screens and main loop (81–115).
5. Arithmetic and multiplication/division: InitEnemies uses MUL for coordinate placement (363–377); PrintNum6At uses DIV (1021–1040).
6. BIOS/DOS interrupts: int 10h/16h/1Ah/21h across rendering, input, timing, exit.
7. String instructions: LODSB for printing strings and sprites (PrintZAt 997–1003, DrawEnemySpriteAt 905–912).
8. Flags and comparisons: bounds checks (UpdateEnemies 412–433, CheckWinLose 642–667).

5 CCA Mapping

CCA1: Complex Problem-Solving

Project is non-trivial: it maintains multiple interacting subsystems (input, timing, bullet management, enemy swarm motion, collision detection, scoring, and win/lose state). Non-linear control flow appears in the main loop (81–115) and nested loops for collisions (553–627) that update shared state (EnemyAlive, BulletActive, Score).

CCA4: Design Constraints (8086 ISA & DOS/BIOS)

Implementation stays inside 8086 constraints: .COM single-segment model (DS=CS), 16-bit registers, and BIOS/DOS interrupts. Rendering uses int 10h helpers (ClearScreen 922–937, PutCharAt 971–987, PrintZAt 989–1010). Timing avoids OS services and uses BIOS tick (FrameSync 168–185).

CCA8: Team Collaboration

Team collaboration is demonstrated through distributed ownership of modules and GitHub history. The contributions table (Section 8) must match meaningful, incremental commits from all members (not last-minute bulk pushes). Each member should be able to explain the procedure(s) they own, for example input/timing vs enemy/collisions vs rendering/HUD.

6 Testing & Results

Testing performed in emu8086 by executing the .asm file and manually validating gameplay behaviors.

Test Case	Input / Action	Expected	Actual
T1	Hold A/Left at x=0	Player stays at x=0	Fill from demo
T2	Hold D/Right at x=79	Player stays at x=79	Fill from demo
T3	Press Space	Bullet appears above player	Fill from demo
T4	Spam Space	Max 5 bullets active	Fill from demo
T5	Wait until top	Bullet deactivates at $y \leq 2$	Fill from demo
T6	Hit enemy	Enemy disappears; Score += 10	Fill from demo
T7	Let enemies descend	Game Over screen	Fill from demo
T8	Destroy all enemies	You Win screen	Fill from demo
T9	Press R on end screen	Game resets to initial state	Fill from demo
T10	Press ESC	Return to DOS	Fill from demo



Figure 1: Gameplay screenshot (emu8086).



Figure 2: Gameplay screenshot (emu8086).

7 Team Contributions

Member Name	Registration No.	Module(s) Owned
Zain Kashif	01-135232-107	Input + Timing (PollInput, FrameSync)
Asim Mahmood	01-135232-054	Enemy Logic + Collisions (UpdateEnemies, CheckCollisions)
Himayat Ullah	01-135232-028	Rendering + HUD (PutCharAt, DrawHUD, DrawEnemies)

8 Challenges & Lessons Learned

1. Flicker/ghosting: Re-drawing every frame in text mode caused trails. Fixed using erase-previous arrays (PrevBullet*, PrevEnemy*, PrevPlayerX) in ErasePrevEntities (723–776) and SyncPrevState (778–816).
2. Timing stability: Tight loops run too fast on emulators. Used BIOS tick (int 1Ah) in FrameSync (168–185).
3. Register pressure: Nested loops (CheckCollisions) required careful push/pop discipline to avoid clobbering state.
4. AI disclosure: This report drafting used AI assistance; the game code was authored/edited by the team with the help of AI (fixing bugs and unexpected behavior).

9 Conclusion

The project delivers a working Space Invaders clone in 8086 assembly under emu8086, demonstrating modular procedure design, array-based state management, BIOS/DOS interrupt usage, and non-trivial control flow. Limitations include text-mode graphics, no sound, and manual testing. Future work can add shields, multiple enemy types, difficulty scaling, and improved rendering.

10 References

References

- [1] R. Brown, *Ralf Brown's Interrupt List*, BIOS/DOS interrupt reference.
- [2] emu8086 Documentation, 8086 assembly IDE/emulator reference.
- [3] Intel Corporation, *8086 Family User's Manual*, instruction set and register model.
- [4] GitHub repository link: https://github.com/D3MON0XXIZZ/CEN323_G_5_SpaceInvaders.